Constraint-Chain Structure in Pokémon Showdown Telemetry

An evaluation-only prototype testing whether structural properties of competitive battle telemetry produce detectable reasoning signal in frontier models

Version 1.0 — implementation spec for Claude Code

> **Hypothesis:** Frontier LLMs exhibit measurably different behavior on real Pokémon Showdown battle chains than on shuffled versions containing the same constraints. The gap widens on longer chains, shrinks as model capability grows, and is driven by the model's ability to exploit causal consistency (HP trajectories, PP depletion, hidden-information reveals) in real sequences that shuffled sequences violate. The experiment is evaluation-only, costs ~$0 on a Claude Max subscription, and is designed to falsify the precondition for a larger training experiment before that training experiment is built.

1. What this document is

This is the complete implementation specification for an evaluation-only prototype. It assumes Claude Code as the execution environment and a human researcher with Python experience as the operator. The spec commits to concrete design decisions (dataset source, translation function, evaluation methodology, statistical tests) so that the experiment can be built in approximately 28 hours of focused work across a long weekend.

The document is structured as: research context (§2), data pipeline (§3), translation function T (§4), chain construction (§5), evaluation methodology (§6), success metrics (§7), methodological risk mitigations (§8), session-by-session build plan (§9), and post-result decision tree (§10). Everything needed to execute is contained here.

2. Research context

2.1 The capability gap being targeted

Current frontier models — Claude Opus 4.7, GPT-5.4, Gemini 3.1 Pro, Kimi K2.6 — are measurably strong on single-turn reasoning and measurably weak on long-horizon agentic tasks. On Terminal-Bench 2.0, SWE-Bench Pro, and Toolathlon, leading models cluster between 50–70% success, with the degradation concentrated at sub-goal transitions and under partial information. This is the capability gap the broader research program targets.

The prototype tests a narrower claim: does game-telemetry-derived constraint structure contain *information a frontier model exploits* when reasoning about adaptation? This is a necessary precondition for any downstream training experiment. If the precondition fails, no training can help. If it holds, the training experiment earns the right to exist.

2.2 Prior work

Three papers establish Pokémon Showdown as a viable LLM research substrate: **Metamon** (2025) trained offline RL agents from Showdown logs that climbed to top 10% of human players; **PokéChamp** (2025) built an expert-level LLM agent using a 3M-game dataset with 500k high-Elo matches; **PokéLLMon** (2024) achieved 49% win rate on the Showdown ladder using GPT-4. All three use Showdown as an environment for *Pokémon-playing agents*.

This prototype asks a different question: whether the *structural properties* of Showdown telemetry — causal consistency, hidden-information reveals, multi-axis resource tracking — produce signal for *domain-general adaptation reasoning*. The distinction matters for novelty: prior work trains agents that play Pokémon; this work tests whether Pokémon logs encode structure that generalizes beyond the domain.

2.3 Why this specific design

The prototype stacks five methodological choices (A through E below) specifically to improve detectability of the effect we care about. Each addresses a different source of noise in the baseline design:

- **A: Long chains (length 20–40).** Short chains (5–15) leave frontier models below their working-memory ceiling; no gap shows because both real and shuffled conditions are trivially handled. Length 20–40 pushes into territory where long-context reasoning errors become measurable.
- **B: Model-capability gradient.** Evaluate on both Claude Haiku 4.5 and Claude Opus 4.7. If telemetry signal is real, weaker models show a larger gap than stronger models. The gradient itself is the scientifically interesting result; a single-model result is less informative.
- **C: Objective scoring via action-match rates.** Replace subjective rubric scoring with "did the model's proposed next action match what high-Elo players did in matching situations?" Computed against the PokéChamp reference distribution. Eliminates scorer-blinding concerns and reduces variance substantially.
- **D1: HP tracking as continuous resource.** The most frequent event class in Showdown logs. Real HP trajectories are causally consistent across turns; shuffled versions frequently aren't. Detecting that inconsistency is precisely the capability we want to measure.
- **E: Asymmetric observability.** Render chains from one player's perspective with their partial information, not ground truth. Real chains have coherent hidden-information reveals; shuffled chains do not. Directly attacks the frontier-model ceiling effect because partial observability is a known weakness.

These five combine to raise the estimated probability of a strong positive result from roughly 15% (baseline design) to roughly 52% (stacked design). The probability estimates are calibrated guesses, not derived from prior experiments.

3. Data pipeline

3.1 Source datasets

Two candidate sources, in order of preference:

1. **PokéChamp dataset.** 3M+ real-player games, 500k+ high-Elo matches, published with the paper at `sites.google.com/view/pokechamp-llm`. Already filtered, parsed, and comes with benchmark structures. Preferred source if accessible.
2. **HolidayOugi/pokemon-showdown-replays on HuggingFace.** Public dataset of Showdown replays via official API, spanning 2005–2026. Fallback if PokéChamp's distribution is gated or missing high-Elo Gen 9 OU.

3.2 Filtering criteria

- **Format:** Gen 9 OU singles only. Largest high-skill population, most stable meta.
- **Skill floor:** both players' Elo ≥ 1700, or tournament matches from ranked events. Below 1700 the decisions are too noisy.
- **Match length:** ≥ 15 turns after exclusion of setup/selection turns. Short matches often indicate resignation or mismatch.
- **Completeness:** match must have `|win|` event and both teams revealed through play, not through forfeit.
- **Patch window:** most recent stable meta (typically 3–4 months of patch version data) to avoid mixing meta eras.

3.3 Target data volumes

Stage	Target count	Purpose
Raw matches downloaded	~2,500	Post-filter yield ~1,000 usable
Chains after translation	~1,500	~1.5 chains/match (perspective variants)
Chains meeting length/structure criteria	~800	Filter to length 20–40 with ≥2 phase transitions
Chains in evaluation set	200 real + 200 shuffled	400 total evaluations per model
Models evaluated	2 (Haiku 4.5 + Opus 4.7)	800 total model evaluations
Seeds per configuration	3	2,400 total scoring events

3.4 Reference action distribution (for Layer C scoring)

Build a lookup table from the full PokéChamp dataset (not just evaluation chains) mapping *(game state signature) → (empirical action distribution among high-Elo players)*. Game state signature is a tuple of (active Pokémon pair, HP brackets, status effects, field conditions, turn number bucket). For each state signature, record the distribution over actions high-Elo players chose in that state. This becomes the ground truth for "optimality" in objective scoring.

State signatures will sometimes be novel (no prior observations). When this happens, back off to broader signatures (drop field conditions, then drop turn bucket, then drop status). Chains where ≥40% of states require maximum backoff are discarded — they can't be objectively scored.

4. Translation function T

4.1 Design principles

T is the central research artifact. Freeze T before evaluation begins. Any changes to T after evaluation starts invalidate the experiment. Four design principles:

1. **Parametric, not hand-crafted per match.** Same function applied uniformly to all matches.
2. **Domain-abstracted.** Output constraint chains contain no Pokémon vocabulary. Pokémon names become `unit_A`, `unit_B`; moves become `action_1`, `action_2`; types, abilities, and items are stripped entirely.
3. **Causally-preserving.** All constraints emitted must be derivable from the prior state plus the current event. No constraint depends on future events.
4. **Inspectable.** A reviewer can read T and understand every mapping without running code.

4.2 Constraint type system

```
ResourceBudget(timestamp, resource, amount, decay, recover_in)
  # continuous or depleting pool: HP, PP, status turns

ToolAvailability(timestamp, tool, state, recover_in)
  # discrete on/off: Pokémon active/fainted, move usable/locked

SubGoalTransition(timestamp, from_phase, to_phase, trigger)
  # regime change: opponent switch, setup completion, critical HP

InformationState(timestamp, observable_added, observable_removed, uncertainty)
  # partial observability: opponent team reveal, ability reveal

CoordinationDependency(timestamp, role, dependency, expected_action)
  # field effects requiring coordinated response: hazards, screens

OptimizationCriterion(timestamp, objective, weight_shift)
  # weather, terrain, boost states that shift optimal play
```

4.3 Event-to-constraint mapping

Table below covers all 12 Showdown protocol events that emit constraints. Events not listed (chat messages, animations, cosmetics, request confirmations) emit nothing.

Showdown event	Condition	Constraint emitted
`|switch|`	player's side	`ToolAvailability(tool=unit_X, state=available)` + previous unit `state=unavailable`
`|switch|`	opponent's side	`SubGoalTransition(from=vs_prior, to=vs_new, trigger=opponent_switch)` + `InformationState(observable_added=[unit_Y])`
`|faint|`	player's side	`ToolAvailability(tool=unit_X, state=unavailable, recover_in=None)` *[permanent]*
`|-damage|`	any	`ResourceBudget(resource=hp_unit_X, amount=new_hp/max, decay=none)`
`|-heal|`	any	`ResourceBudget(resource=hp_unit_X, amount=new_hp/max, decay=none)`
`|-status|`	par, brn, slp, frz, psn, tox	`ResourceBudget(resource=status_unit_X, amount=severity, decay=monotone_decrease_if_turn_based)`

`|-boost| / |-unboost|`	stat changes	`ResourceBudget(resource=boost_stat_unit_X, amount=stage/6.0, decay=none)`
`|move|`	PP depletion hit	`ResourceBudget(resource=pp_action_M, amount=remaining/max, decay=none)`
`|-sidestart|`	Stealth Rock, Spikes, Sticky Web, Toxic Spikes	`CoordinationDependency(role=field_side_N, dependency=hazard_type)`
`|-weather|`	rain, sun, sand, snow	`OptimizationCriterion(objective=weather_state, weight_shift=type-specific)`
`|-fieldstart|`	terrain or trick room	`OptimizationCriterion(objective=field_state, weight_shift=terrain-specific)`
`|turn|`	every 5 turns	`ResourceBudget(resource=match_time_remaining, amount=1.0 - turn/60.0, decay=monotone_decrease)`

4.4 Rendering: chain → abstract prompt

Constraints are rendered as abstract English with no Pokémon vocabulary. Pokémon species become `unit_A` through `unit_F` (stable within a chain). Moves become `action_1` through `action_4` per unit. Field effects are generic: "hazard_X on side 1", "weather_rain for 5 steps". This is critical for preventing contamination from the model's Pokémon knowledge.

```
Example rendered chain step:
"Step 14 (t=280s):
  ToolAvailability: unit_A is now UNAVAILABLE (permanent)
  SubGoalTransition: phase shifted from 'active_A_vs_B' to 'forced_switch_required'"
```

4.5 Asymmetric observability (component E)

Chains are rendered from *one player's perspective*, not ground truth. Player A's perspective:

- Sees Player A's units, HP, moves, status — full visibility
- Sees Player B's units *only after they've been switched in*. Before first switch, rendered as `unit_unknown_slot_K`
- Sees Player B's move-sets only after moves are used. Unused moves remain hidden
- Sees Player B's HP at bucket granularity (100%, 75%, 50%, 25%, faint) — the granularity Showdown shows spectators

This partial-information rendering is what makes asymmetric observability work. Real chains reveal information in causally-consistent order (units appear when switched in, moves appear when used). Shuffled chains reveal information in random order — information appears, disappears, reappears. This inconsistency is detectable reasoning signal.

5. Chain construction

5.1 Per-match chain yield

Each match produces up to 2 chains (one per player perspective). Target both perspectives when both players meet the Elo filter; otherwise take only the qualifying perspective. Expected yield ~1.5 chains per match after filtering.

5.2 Chain validity filter

A chain is valid if *all* of the following hold:

- Length between 20 and 40 constraints (inclusive)
- Contains ≥2 `SubGoalTransition` events (regime changes)

- Contains ≥1 `ToolAvailability` with `state=unavailable` (something became unusable)
- Contains ≥10 `ResourceBudget` events (dense HP/PP/status signal)
- Timestamps are strictly non-decreasing
- No two consecutive constraints are identical (no stuttering events)

5.3 Shuffling (control condition)

For each real chain, produce one shuffled variant:

- Permute constraints uniformly at random (seeded for reproducibility)
- Reassign timestamps to preserve monotonic ordering (so shuffled chains don't reveal themselves via out-of-order timestamps)
- Preserve `InformationState` semantics — if a unit was "observable_added" in a later turn in the original, the shuffled version keeps that as the same event, just moved to an earlier position. This is deliberate: shuffled chains should violate causal consistency, which is the signal we're measuring

Chain IDs are suffixed with `_shuffled_{seed}` for traceability. Real and shuffled chains from the same match share a `match_id` field but are treated as independent samples in analysis.

6. Evaluation methodology

6.1 Evaluation task

For each chain, the model is given:

- A generic system prompt describing an "adaptive pipeline under shifting constraints"
- The rendered chain (real or shuffled), step by step, up to some cutoff step K (typically step 15 in a length-30 chain)
- A prompt asking: "given the state at step K, propose the next action for the agent at step K+1"

The model's response is scored against two metrics (§7). The system prompt is fixed across all chains and contains no Pokémon references.

6.2 Prompt template (fixed, versioned)

```
You are reasoning about an adaptive pipeline that operates under
a sequence of changing constraints. At each step, new constraints
may appear, existing constraints may change, and resources may
be depleted. Your job is to propose the correct adaptation at
each step, given the full prior history.

At each step you receive a partially-observable state: you see
what happened to YOUR units but only limited information about
opposing units until they are revealed through action.

Constraints carry forward unless explicitly superseded. Treat
"tool UNAVAILABLE" as persistent unless a later event makes it
available again.

---

[Rendered chain steps 1 through K]

---

Given the state above, propose the next action for your side at
step K+1. Output only the action label (e.g., "use action_2 with
unit_A" or "switch to unit_C"). No explanation.
```

6.3 Sampling

- Temperature: 0.0 (deterministic)
- Max tokens: 50 (force concise responses)
- 3 seeds per configuration for variance estimation (seeds 42, 1337, 7919)
- One model at a time; never interleave models in the same batch to avoid carryover artifacts

6.4 Scorer blinding

Scoring happens in a separate Claude Code session from generation. The scoring session receives only: the chain, the cutoff step K, and the model's proposed action. It does not receive: which model produced the action, whether the chain is real or shuffled, or which seed was used. All metadata is stripped before scoring.

In practice, objective scoring (§7 Layer 1) is fully automated via the reference action distribution — no LLM scorer needed. Subjective rubric scoring (§7 Layer 2) is used only as a secondary check on edge cases and requires the blinding described above.

7. Success metrics (three-layer framework)

7.1 Layer 1 (primary): Objective action-match rate

For each evaluation, look up the game state at step K+1 in the PokéChamp reference distribution. Let `P(a | state)` be the empirical probability that high-Elo players chose action `a` in that state. The model's score for that evaluation is:

```
top_k_match = 1 if model_action ∈ top_k(P(· | state)) else 0
  where k = 3 (top-3 action match)

probability_mass = P(model_action | state)
  # how much of the empirical mass the model's choice captures
```

Primary reported metric: mean top-3 match rate, computed separately for real vs shuffled chains, separately for each model (Haiku vs Opus), across 3 seeds.

Pre-registered success threshold: real-vs-shuffled gap ≥ 0.05 (5 percentage points) on top-3 match rate, with $p < 0.05$ via two-sample proportion test, on *at least one* of the two models. Both models showing a gap is stronger but not required.

7.2 Layer 2: Legality vs optimality decomposition

For each evaluation, compute two independent metrics:

- **Constraint satisfaction rate (legality).** Fraction of chain steps where the model's action is permitted by active constraints. A fainted unit cannot act; an unavailable tool cannot be used. Computed objectively from chain state.
- **Optimality proxy.** Among legal actions, probability mass captured in the reference distribution (as in Layer 1).

Report the *product* of the two as the composite "coupled" metric. A model that games either axis alone (always picks safe action; always mimics top player without checking legality) will score high on one and low on the other. High composite score requires both.

Layer 2 success: gap between real and shuffled on the composite metric ≥ 0.04, consistent with Layer 1 gap direction.

7.3 Layer 3: Subset diagnostic analysis

After Layers 1 and 2, break down the effect by:

- **Chain length:** 20–25, 26–32, 33–40. Is the effect larger on longer chains? (Predicted yes.)
- **Model capability:** is the gap larger on Haiku than Opus? (Predicted yes.)
- **Constraint-type composition:** chains dominated by `ResourceBudget` vs chains dominated by `SubGoalTransition`. Does telemetry help one type more?
- **Archetype:** aggressive (fast-paced, many KOs), stall (resource attrition), setup (boost-focused). Does effect hold across archetypes or only some?
- **Turn of cutoff K:** early-chain cutoffs (step 10 of 30) vs late-chain cutoffs (step 25 of 30). Does the effect grow as the chain gets longer before cutoff?

7.4 Outcome tiers

Outcome	Layer 1 result	What it earns
Strong positive	Gap ≥ 0.08 with $p < 0.01$; consistent across subsets in Layer 3; Haiku gap > Opus gap	Publishable preliminary result; justifies v2 training experiment
Moderate positive	Gap 0.05–0.08, $p < 0.05$, mixed Layer 3	Larger-n replication (500 chains); not yet publishable
Weak / mixed	Gap < 0.05 or inconsistent direction across models	Diagnostic value; suggests reframing hypothesis
Null	Gap near zero on both models	Publishable negative; kills v3 training proposal as scoped
Reversed	Shuffled > real	Most interesting; suggests working-memory limitation rather than structure exploitation

Target probability distribution for this stacked design: strong positive ~40%, moderate ~20%, weak/mixed ~17%, null ~20%, reversed ~3%. These are calibrated guesses.

8. Methodological risk mitigations

8.1 Contamination (most serious risk)

If rendered chains leak Pokémon vocabulary, the model uses Pokémon priors instead of constraint reasoning. Mitigations:

- Abstract rendering layer is a single fixed module; every chain goes through it; unit-test it against a Pokémon name dictionary to ensure zero leakage
- Spot-check 20 rendered chains manually before evaluation begins; any Pokémon vocabulary visible = implementation bug
- Validation prompt: give the rendered chain to Claude and ask "can you identify the source domain?" If it guesses Pokémon confidently, rendering is too leaky

8.2 Reference distribution sparsity

If too few chain states have reference data, Layer 1 scoring becomes unreliable. Mitigations:

- Pre-compute reference distribution over the full PokéChamp dataset (3M games), not just evaluation chains
- Implement state-signature backoff (§3.4); measure backoff depth per evaluation and report
- Discard chains where ≥40% of scored states require maximum backoff

8.3 Shuffle artifact detection

A clever model might detect "this chain has a constraint violation, so it's shuffled, so I should guess differently." If it does, the real-vs-shuffled gap is an artifact of detection not of exploitation. Mitigations:

- Measure whether model responses to shuffled chains are "defensive" (high variance, low-commitment actions). If yes, the effect is partly detection
- Include a third condition: "semi-shuffled" chains where only the last 3 steps are shuffled. If detection drives the effect, semi-shuffled should look mostly like real; if exploitation drives it, semi-shuffled should look partly degraded

8.4 Model-specific artifacts

Haiku and Opus might disagree not because of capability but because of different training distributions. Mitigations:

- Report absolute performance on each model, not just the gap
- If possible, include one open-weight model (e.g., Qwen 2.5 32B) as a sanity check against Anthropic-specific artifacts

8.5 Selection bias in high-Elo filtering

High-Elo chains may be systematically different from median play in ways that make them easier for the model (cleaner, more "optimal" structure). This is fine for our research question but should be reported. Do not claim the effect generalizes to low-Elo play without testing it.

9. Session-by-session build plan

The build is structured for Claude Code sessions, each 2–4 hours of focused work. The total is ~28 hours across 8 sessions, feasible across a long weekend plus an additional evening.

Session 1: Environment + data acquisition (3 hours)

Goal: Download dataset, verify format, set up project structure.

- Scaffold Python project: `pyproject.toml`, `requirements.txt`, `.env.example`
- Download PokéChamp dataset (preferred) or HuggingFace Showdown replays (fallback)
- Verify at least 2,500 Gen 9 OU matches with both players Elo ≥ 1700
- Build a smoke-test parser that reads one match and extracts event stream
- **Gate:** single match successfully parsed end-to-end, event count non-zero, event types distributed sensibly

Session 2: Reference action distribution (4 hours)

Goal: Build the state-signature → action distribution lookup from the full dataset.

- Process ≥100k high-Elo games through state-signature extraction
- Build lookup table: state signature → action distribution
- Implement state-signature backoff logic
- Serialize to disk (Parquet or pickle)
- **Gate:** ≥80% of randomly-sampled states from eval chains return a non-backoff reference distribution

Session 3: Translation function T (4 hours) — research artifact

Goal: Implement and freeze T.

- Implement all 12 event-to-constraint mappings from §4.3
- Write unit tests for each mapping against fixture events
- Implement abstract rendering layer (§4.4)
- Manual inspection of 20 rendered chains for Pokémon vocabulary leakage
- Freeze T: tag git commit as `T-v1.0-frozen`, do not modify after this point
- **Gate:** all unit tests pass; manual inspection shows zero Pokémon vocabulary in rendered output

Session 4: Chain construction + validity filter (3 hours)

Goal: Convert 2,500 matches into ~400 eval-ready chains.

- Implement asymmetric observability rendering (§4.5)
- Implement shuffler with timestamp preservation (§5.3)
- Implement chain validity filter (§5.2)
- Process all matches, produce `chains/real/*.jsonl` and `chains/shuffled/*.jsonl`
- Compute distribution statistics: chain length histogram, constraint type frequencies, subset labels
- **Gate:** ≥200 valid real chains, matching shuffled variants, distribution statistics look sensible

Session 5: Evaluation prompts + sampling infrastructure (3 hours)

Goal: Build the evaluation runner.

- Implement prompt builder using fixed template (§6.2)
- Implement sampling runner supporting both Claude Haiku 4.5 and Claude Opus 4.7 via API
- Implement seed control (seeds 42, 1337, 7919)
- Dry-run on 5 chains to verify response format and latency
- **Gate:** 5 test evaluations complete end-to-end with responses parsed cleanly

Session 6: Main evaluation run (5 hours, mostly wait time)

Goal: Run 2,400 evaluations (400 chains × 2 models × 3 seeds).

- Launch batch; monitor for errors, rate limits, malformed responses
- Save all responses to `results/raw/*.jsonl` with full metadata (except stripped for scorer)
- Mid-run sanity check at 20% completion: response format is parseable, no systematic failures
- **Gate:** ≥95% of evaluations produce parseable responses

Session 7: Scoring + statistical analysis (4 hours)

Goal: Compute Layer 1, 2, and 3 metrics with proper statistics.

- Layer 1: compute top-3 match rate per chain, aggregate by (model, condition, seed)
- Layer 2: compute legality rate, optimality proxy, and coupled product
- Layer 3: break down by subset (length, archetype, composition, cutoff)
- Statistics: two-sample proportion test for Layer 1, Welch's t-test for Layer 2, report confidence intervals
- Generate results notebook with all tables and plots
- **Gate:** all three layers have computed statistics with confidence intervals; outcome tier is clear

Session 8: Writeup (2 hours)

Goal: Draft the 6–10 page technical report.

- Populate template with actual numbers, tables, and plots
- Write limitations section honestly; do not claim more than the data supports
- Commit findings, code, and data to a git repo

10. Post-result decision tree

Different outcomes trigger different next actions. All of these paths are honest — including the paths where the program stops.

10.1 If Strong Positive

Write up a 6–10 page technical report with methodology, results, and limitations. Share with 3–5 internal ML-adjacent colleagues for technical feedback before broader distribution. Do not make public claims. Use the report to seek sponsorship for the v2 training experiment ($5–10k, 4–6 weeks, single-game, 7B model with LoRA).

10.2 If Moderate Positive

Run a larger-n replication: 500 chains per condition instead of 200, same methodology. Additional cost is minimal (a few more hours on Claude API). Report both results honestly.

10.3 If Weak / Mixed

Diagnostic analysis: which Layer 3 subset drove the signal, if any? If long chains show signal but short don't, the hypothesis is partially confirmed and the framing tightens. If no subset shows signal, treat as Null.

10.4 If Null

Publish the negative result (blog post or short technical note on arXiv). This is valuable — it saves other researchers from running the same experiment. Do not proceed to v2 training as scoped. Consider reframing the hypothesis entirely.

10.5 If Reversed

Most scientifically interesting outcome. Write up as a finding about working-memory limitations under structured sequences, not as a telemetry-training claim. Completely different research direction.

11. What this prototype does NOT prove

- It does not prove that training on telemetry helps. It tests whether telemetry contains detectable structure. Different claim.
- It does not prove the effect generalizes beyond Pokémon Showdown. Multi-source validation requires Phase 2.
- It does not prove the effect scales to larger models. Scaling study requires Phase 3.
- It does not prove the effect is driven by sequential structure specifically (could be causal consistency, could be information coherence, could be something else). Mechanism analysis requires follow-up.
- It does not justify public claims about "AI trained on Pokémon." Reserve that framing for results that actually support it.

12. What counts as success for this build

The prototype succeeds as an engineering artifact if:

1. The pipeline runs end-to-end on 400 chains and produces parseable results.
2. Methodology is tight: blinding is clean, abstraction prevents contamination, statistics are proper.
3. Results — whatever they show — are trustworthy enough to make a sponsorship decision on.

The prototype succeeds as a research contribution if Layer 1 shows a statistically significant real-vs-shuffled gap on at least one model. The prototype succeeds as a scientific contribution regardless of direction: positive justifies further investment, null is publishable, mixed is diagnostic.

This specification is frozen at v1.0. Methodological changes during execution invalidate the pre-registration. Any divergence from this spec must be recorded and reported in the final writeup. Estimated total build time: 28 hours across 8 sessions. Estimated total cost: $0–20 in Claude API overage if subscription quota is exceeded. Probability of strong positive result: ~40%, calibrated guess.